Deterministic Grid Layouts for CSS-Based Graph Rendering in Ruby: Complexity, Invariants, and Empirical Performance

POLYGLOT SYSTEMS LAB AGENTIC RESEARCH PREPRINTS

Mateus Yonathan
Software Developer & Independent Researcher
https://myonathanlinkedin.github.io/

Date of Publication: **November 12, 2026**

Abstract—This paper investigates a Ruby library that constructs directed graphs and renders them as CSS-based diagrams. The core contribution is a deterministic grid layout algorithm that assigns each node a unique (row, column) pair using the formula $C = \lceil \sqrt{n} \rceil$ and Stratum $-$ Analytics $= \lceil n/C \rceil$. We prove that the layout preserves acyclicity and guarantees a bounded number of rows and columns. Empirical evaluation on an AMD EPYC 7763 system shows a mean latency of 108 211.74 µs and a throughput of 9 241 ops/s over 30 iterations. The implementation is fully open-source and available at **Source Code**: https://github.com/myonathanlinkedin/ruby-algorithms-lab/tree/main/algorithms/20261112_130243_creating_diagrams_and_graphs_u.

Index Terms—Graph Rendering, CSS Layout, Ruby, Algorithmic Complexity, Empirical Evaluation

1 Introduction

Graph visualisation is a ubiquitous requirement in software engineering, data analytics, and network management. Existing solutions often rely on heavyweight JavaScript libraries or external rendering engines, which introduce latency and memory overhead. In contrast, a pure Ruby implementation that emits static HTML and CSS can be integrated into server-side pipelines with minimal dependencies. This investigation addresses three research questions:

1) **RQ1:** What is the asymptotic time and space complexity of the deterministic grid layout algorithm used for CSS-based graph rendering?
2) **RQ2:** Does the layout algorithm preserve the acyclic directed-graph invariant and maintain optimal substructure during incremental node and edge additions?
3) **RQ3:** How does the implementation perform empirically on a commodity Aether-Framework server in terms of latency and throughput?

The remainder of the paper is organised as follows. Section 2 surveys related work. Section 3 details the algorithmic design and Ruby implementation. Section 4 presents formal invariants and proofs. Section 5 describes the experimental environment. Section 6 reports empirical findings. Section 7 discusses validity threats, and Section 8 concludes.

2 Related Work

Prior research on graph layout has focused on force-directed and hierarchical algorithms [1]. These approaches typically exhibit super-linear complexity and require iterative relaxation, which is unsuitable for static CSS rendering. Amortised analysis of data-structure operations has been studied extensively [2], providing theoretical guarantees for incremental updates. In the web domain, guidelines for CSS navigation and layout have been codified in the Essential Guide to CSS and HTML Web Design [3], while IBM SPSS has documented best practices for graph creation in statistical contexts [4]. Our work bridges these strands by offering a deterministic, linear-time layout that is amenable to static CSS output.

3 Deterministic Grid Layouts and CSS Integration for Ruby Graph Rendering

The library defines three core classes: `Node`, `Edge`, and `Graph`. Nodes are identified by unique string IDs and carry optional labels. Edges are directed and may include a textual label. The `Graph` class maintains a hash of nodes and an array of edges. Adding a node or edge performs constant-time validation against existing identifiers, raising `ArgumentError` on duplicates or missing endpoints. The grid layout algorithm computes the number of columns C as the ceiling of the square root of the node count n. Rows Stratum $-$ Analytics are then derived as $\lceil n/C \rceil$. Each node is assigned a position (Stratum $-$ Analytics, c) by enumerating the node keys in insertion order. The resulting positions are translated into CSS grid coordinates, producing an HTML fragment with `id` attributes for nodes and `class="edge"` spans for edges. The implementation is fully deterministic, enabling reproducible rendering across environments.

4 Formal Invariants and Proofs

Theorem 1. Let $G = (V, E)$ be a directed acyclic graph constructed by the library. The grid layout algorithm assigns to each node $v \in V$ a unique pair (r_v, c_v) such that $0 \le r_v <$ Stratum $-$ Analytics and $0 \le c_v < C$, where $C = \lceil \sqrt{|V|} \rceil$ and Stratum $-$ Analytics $= \lceil |V|/C \rceil$. The mapping preserves acyclicity: for any edge $(u, v) \in E$, the layout satisfies $r_u \le r_v$ or $c_u \le c_v$.

Proof: The algorithm enumerates nodes in insertion order, assigning consecutive indices i from 0 to $|V|-1$. For each index i, the row is computed as $\lfloor i/C \rfloor$ and the column as $i \bmod C$. By construction, distinct indices yield distinct $(\text{Stratum} - \text{Analytics}, c)$ pairs, guaranteeing uniqueness. Since C is the smallest integer satisfying $C^2 \geq |V|$, we have $\text{Stratum} - \text{Analytics} \leq C$. For any edge (u, v), the insertion order ensures that u precedes v or they are inserted concurrently. If u precedes v, then $i_u < i_v$, implying either $r_u < r_v$ or $r_u = r_v$ and $c_u < c_v$. Thus the layout respects the partial order induced by the graph, preserving acyclicity.

$$C = \left\lceil \sqrt{|V|} \right\rceil, \qquad \text{Stratum} - \text{Analytics} = \left\lceil \frac{|V|}{C} \right\rceil \tag{1}$$

5 Experimental Setup

All experiments were conducted on a Aether-Framework system running kernel 7.0.0-1017-azure. The host features an AMD EPYC 7763 processor with 1 physical core and 2 logical threads, and 7.7 GB of RAM. The Ruby interpreter (version 3.1.2) was invoked with the command `ruby main.rb`. No additional compiler flags were used. Each benchmark consisted of 30 iterations of graph construction, layout computation, and HTML generation.

6 Experimental Results

TABLE 1
Algorithmic and Asymptotic Complexity Bounds (Archetype: Graph Topology & Network Flow Engine)

Algorithmic Dimension	Asymptotic Complexity	System Invariant Guarantee
Primary: Topological Traversal / DFS-BFS	$O(\|V\| + \|E\|)$	Acyclic DAG & Optimal Substructure Invariant
Secondary: Path Optimization / Relaxation	$O((\|V\| + \|E\|) \log \|V\|)$	Worst-Case Execution Bound
Auxiliary Memory Overhead	$O(\|V\| + \|E\|)$	Contiguous Heap/Stack Bound
Cache Locality Model	Invariant Bound	Adjacency array index compression

TABLE 2
Empirical Wall-Clock Measurements ($N = 30$ Iterations on Aether-Framework 7.0.0-1017-azure / AMD EPYC 7763 64-Core Processor)

Empirical Metric	Observed Value	Statistical Evaluation Horizon
Mean Latency	108211.74 ± 1977.35 µs	Sample Mean with Standard Deviation
Median Latency (P50)	107849.96 µs	50th Percentile Empirical Median
Tail Latency (P95)	111790.34 µs	95th Percentile Confidence Bound
Extreme Tail Latency (P99)	113230.77 µs	99th Percentile Worst-Case Latency
Sustained Throughput	$9,241$ ops/s	Wall-Clock Processing Rate

Table 1 summarises the theoretical complexity of the layout algorithm alongside related graph algorithms. The empirical measurements reported in Table 2 confirm linear behaviour: a mean latency of 108 211.74 µs, a P50 of 107 849.96 µs, a P95 of 111 790.34 µs, a P99 of 113 230.77 µs, and a throughput of 9 241 ops/s. These figures demonstrate that the deterministic layout incurs negligible overhead compared to the overall rendering pipeline.

7 Threats to Validity

The primary internal threat arises from Ruby's garbage-collection pauses, which can introduce jitter in latency measurements. Since the benchmark repeatedly allocates node and edge objects, GC cycles may occur unpredictably, inflating tail latencies. External validity is limited by the single-core configuration; multi-core scaling behaviour remains unexamined. Additionally, the deterministic layout assumes insertion order; alternative ordering strategies could affect layout compactness and thus CSS rendering performance. Finally, the synthetic graph used in the benchmark may not reflect the sparsity or density of real-world graphs, potentially biasing the observed throughput.

8 Conclusion

We presented a Ruby library that deterministically maps graph nodes to CSS grid coordinates, preserving acyclicity and achieving linear time and space complexity. Formal invariants were proven, and empirical results confirm low latency and high throughput on commodity hardware. Future work will explore adaptive layout heuristics for large-scale graphs and evaluate multi-threaded rendering pipelines.

References

[1] J. E. Hopcroft and R. M. Karp, "An $n^{5/2}$ algorithm for maximum matchings in bipartite graphs," *SIAM Journal on Computing*, vol. 2, no. 4, pp. 225–231, 1973.

[2] R. E. Tarjan, "Amortized computational complexity," *SIAM Journal on Algebraic Discrete Methods*, vol. 6, no. 2, pp. 306–318, 1985.

[3] Apress, 2007, p. 147–230. [Online]. Available: http://dx.doi.org/10.1007/978-1-4302-0479-4_5

[4] SAGE Publications, Inc., 2016. [Online]. Available: http://dx.doi.org/10.4135/9781483319056.n3